

Predicting Steam User Reception Using Metadata, Engagement Signals and Multilabel Tags

Stefano Triscali

July 24, 2026

Abstract

This technical report investigates whether Steam game metadata and post-release engagement signals can predict user reception. A Bayesian-smoothed review reception score is constructed from positive and negative Steam reviews to reduce instability for games with few reviews. Numerical metadata, genres, categories and tags are used to train and compare several regression models. XGBoost outperforms linear baselines, while multilabel tags provide the strongest categorical improvement. The final model achieves an RMSE of approximately 17.38 and an R^2 of 0.317 on a held-out test set. Error analysis shows that extreme user reception remains difficult to predict, suggesting that additional qualitative information such as review text, developer reputation and community sentiment could further improve performance.

1 Introduction and Research Question

This project builds on my bachelor’s thesis, *The Sunk Cost Fallacy in Cultural Production and Consumption: Evidence from the Creative Industries and Applications to the Video Game Sector*. After completing that work, I wanted to apply the tools and methods learned in the Applied Machine Learning course to a domain that I had already explored from an economic and cultural perspective: the video game industry.

Machine learning is widely used in digital markets, especially for recommendation systems, user profiling, demand forecasting and marketing analytics. This motivated my interest in investigating whether user reception in the video game sector can be modelled using data-driven methods.

Steam was chosen as the platform of analysis because it hosts a large and diverse catalogue of games, ranging from small independent titles to major commercial releases. User reception varies widely across games and is likely influenced by multiple factors, including gameplay characteristics, audience expectations, visibility, technical quality, community dynamics and post-release engagement.

The main research question is therefore:

Can Steam game metadata and post-release engagement signals be used to predict user reception?

The project is framed as a post-release reception modelling task rather than a pre-release success forecasting task. This distinction is important because several predictors used in the analysis, such as estimated ownership, playtime indicators and peak concurrent users, are observed only after release.

2 Dataset

The empirical analysis is based on the Steam Games Dataset published by Roman (2022). The dataset contains game-level metadata from Steam, including identification variables, release information, platform availability, price-related variables, user review counts, playtime indicators, developers, publishers, genres, categories and user-defined tags. Starting from this raw source, I built a cleaned version of the dataset to obtain a more stable and suitable input for the machine-learning workflow developed in this project.

The raw dataset was downloaded in Google Colab through the `kagglehub` library using the Kaggle dataset identifier `fronkongames/steam-games-dataset`. Since the imported CSV file did not directly provide the desired column structure, I manually assigned the correct column names before loading the file into a pandas DataFrame. The initial set of variables included, among others, `AppID`, `Name`, `Release date`, `Estimated owners`, `Peak CCU`, `Required age`, `Price`, `Discount`, `DLC count`, `Supported languages`, `Windows`, `Mac`, `Linux`, `Metacritic score`, `User score`, `Positive`, `Negative`, `Achievements`, `Recommendations`, `Average playtime forever`, `Developers`, `Publishers`, `Categories`, `Genres` and `Tags`.

The cleaning procedure was designed to remove observations and variables that could introduce noise or reduce the interpretability of the modelling task. First, the variables `Movies` and `Score rank` were removed from the dataset. The former mainly contains media-related information that was not used in the predictive task, while the latter was not retained as a modelling feature. Second, games with missing or empty names were excluded, since the game title is a fundamental identifier for checking the validity of each observation. Third, I removed titles containing terms such as *playtest*, *beta* or *demo*. These entries often refer to non-final or limited versions of games and may not be comparable with complete commercial releases. Finally, duplicate observations were removed using the pair `Name-Developers` as the reference key, in order to avoid counting the same game more than once when both the title and the developer were identical.

After this preprocessing step, the cleaned dataset was exported as `steam_cleaned_2026.csv` and published as a separate Kaggle dataset (Triscali, 2026). This cleaned file was then mounted again in Google Colab and used as the starting point for the rest of the project. In this way, all subsequent steps, including feature engineering, target construction, model training and model evaluation, were based on a fixed and reproducible version of the data.

The final dataset therefore represents a game-level tabular dataset in which each row corresponds to a Steam title and each column describes either numerical, categorical or textual metadata associated with that title. Numerical variables include, for example, price, discount, peak concurrent users, age requirement, number of DLCs, number of achievements, playtime measures and review-related counts. Categorical variables include platform availability, developers, publishers, genres, categories and tags. These variables are particularly suitable for the objective of the project because they combine commercial, technical and community-related information that may help explain or predict user reception on Steam.

3 Target Construction

The objective of the project is to predict user reception for Steam games. Since the dataset does not contain a direct Net Promoter Score, I constructed a proxy target using the number of positive and negative user reviews available for each game. This choice is inspired by the logic of the Net Promoter Score (Reichheld, 2003), where the final score is obtained by comparing favourable and unfavourable user evaluations. In the Steam context, positive reviews can be interpreted as favourable user feedback, while negative reviews represent unfavourable reception. However, this measure should be understood as an NPS-inspired proxy rather than a real survey-based NPS, since Steam reviews are binary and do not include the traditional promoter, passive

and detractor categories.

For each game i , I first computed the total number of reviews as:

$$n_i = \text{Positive}_i + \text{Negative}_i$$

where Positive_i and Negative_i are respectively the number of positive and negative reviews received by game i . Games with no reviews were excluded from the modelling dataset, since no reliable user reception signal could be derived for them. After this filtering step, the dataset contained 82,766 observations.

A first raw review score was computed as:

$$\text{review_score_raw}_i = \frac{\text{Positive}_i - \text{Negative}_i}{\text{Positive}_i + \text{Negative}_i} \times 100$$

This score ranges from -100 to 100 . A value close to 100 indicates that almost all reviews are positive, while a value close to -100 indicates that almost all reviews are negative. A value around zero indicates a balance between positive and negative reviews.

Although this raw score is intuitive, it can be highly unstable for games with very few reviews. For example, a game with one positive review and zero negative reviews would obtain a raw score of 100 , even though this value is based on only one observation. This would make very small games appear as extreme cases, despite the limited amount of evidence available.

To reduce this problem, I applied a Bayesian smoothing approach inspired by the weighted rating formula used by IMDb for its Top Rated charts (IMDb, n.d.). IMDb combines the average rating of a title with the mean rating across all titles, weighting the two components according to the number of ratings received by the title. In this project, I adapted the same logic to Steam’s binary positive/negative review structure.

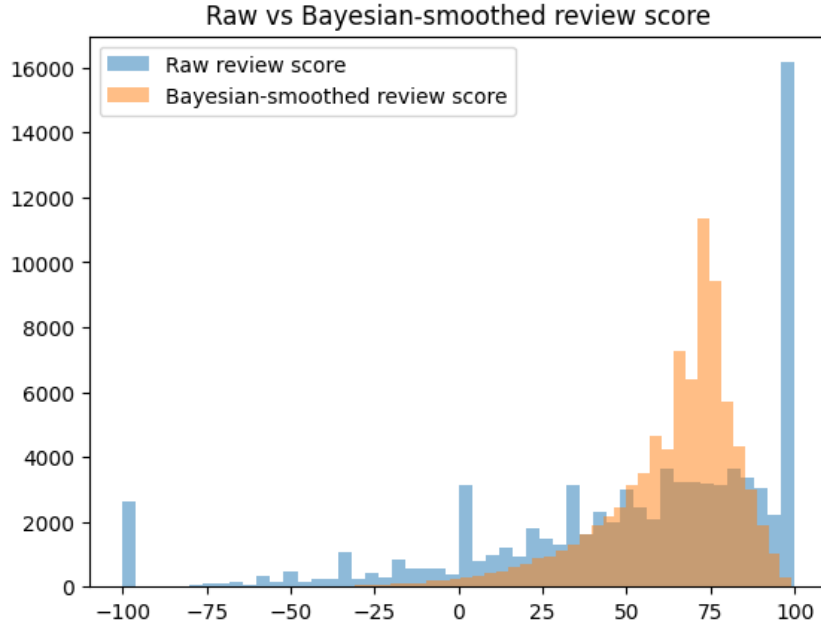


Figure 1: Distribution of the raw review score and the Bayesian-smoothed review score. Bayesian smoothing reduces the concentration of extreme values while preserving the overall structure of the distribution.

First, I defined the observed review balance for each game as:

$$R_i = \frac{\text{Positive}_i - \text{Negative}_i}{\text{Positive}_i + \text{Negative}_i}$$

Then, I computed the global review balance across the whole dataset as:

$$C = \frac{\sum_i \text{Positive}_i - \sum_i \text{Negative}_i}{\sum_i (\text{Positive}_i + \text{Negative}_i)}$$

The smoothing parameter m was set equal to the median number of total reviews in the dataset, which was 23. Unlike IMDb, where m represents the minimum number of ratings required to enter the Top Rated chart, here m is used as a data-driven shrinkage parameter. It determines how strongly games with few reviews are pulled toward the global average.

The final Bayesian-smoothed review score was computed as:

$$\text{review_score_bayes}_i = \left(\frac{n_i}{n_i + m} R_i + \frac{m}{n_i + m} C \right) \times 100$$

Equivalently, this can be written as:

$$\text{review_score_bayes}_i = \frac{(\text{Positive}_i - \text{Negative}_i) + mC}{n_i + m} \times 100$$

This transformation shrinks the raw score of each game toward the global average review balance. The shrinkage is stronger for games with few reviews and weaker for games with many reviews. Therefore, games with limited review evidence are prevented from receiving excessively extreme scores, while games with a large number of reviews remain close to their observed raw score.

A related alternative would have been to use the lower bound of the Wilson score confidence interval, as suggested by Evan Miller (2009) for ranking items with positive and negative ratings. This approach is particularly useful when the goal is to rank items conservatively, because it penalizes scores based on the uncertainty generated by small sample sizes. However, in this project the objective is not to build a top-ranked list of games, but to construct a continuous regression target representing expected user reception. For this reason, Bayesian smoothing was preferred, as it provides a direct and interpretable adjusted score while still correcting the instability of raw averages.

The final regression target used in the project is therefore `review_score_bayes`. The raw score was kept only for comparison and diagnostic purposes, while the Bayesian-smoothed score was used as the dependent variable in all regression models.

Finally, since the variables `Positive`, `Negative` and `total_reviews` are directly involved in the construction of the target, they were not used as predictive features in the modelling pipeline. This avoids giving the model direct access to the same information used to define the outcome variable.

4 Methodology

The project is framed as a supervised regression problem. The goal is to learn a function that maps Steam game metadata into a continuous measure of user reception, represented by the Bayesian-smoothed review score constructed in the previous section. Formally, for each game i , the model receives a vector of predictors X_i and estimates:

$$\hat{y}_i = f(X_i)$$

where y_i denotes the target variable `review_score_bayes`. Since the target is continuous and is derived from a review score originally bounded between -100 and 100 , regression models were used rather than classification models.

It is important to clarify that this is a post-release prediction task, not a pre-release forecasting task. Some of the predictors used in the project, such as estimated owners, average playtime, median playtime and peak concurrent users, are only observable after a game has been released.

Therefore, the objective is not to predict commercial or user success before launch, but to evaluate whether available post-release metadata and engagement indicators can explain and predict user reception.

No feature-selection or hyperparameter-selection decision was based on held-out test performance.

Figure 2 summarizes the complete modelling workflow. All feature-selection and model-selection decisions were performed using the training set, while the held-out test set was reserved for final evaluation.

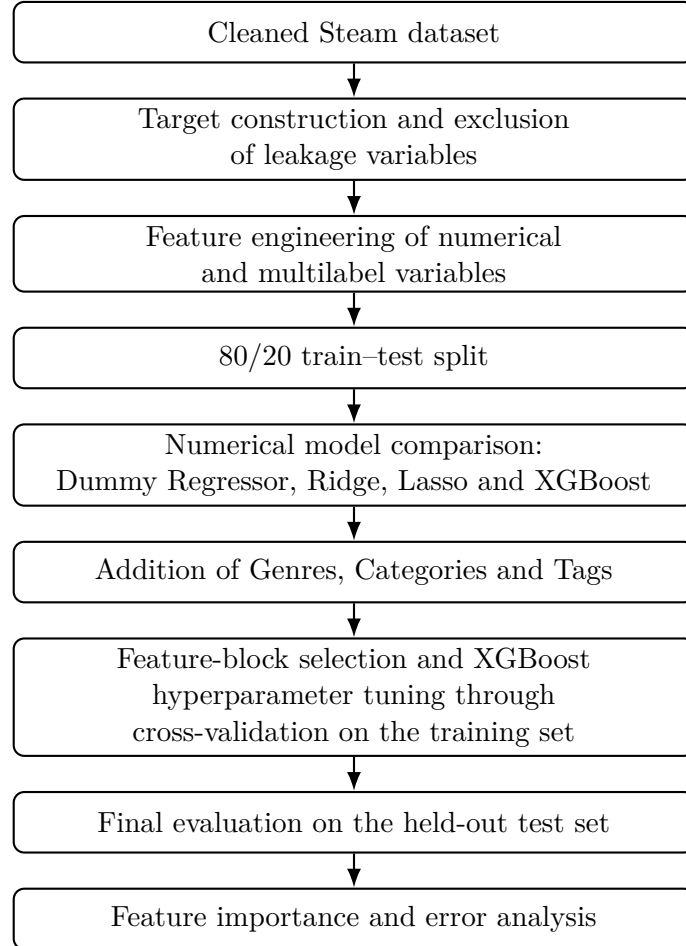


Figure 2: Overview of the machine-learning workflow used in the project.

4.1 Feature Engineering

Starting from the cleaned Steam dataset, I constructed a set of numerical and categorical predictors. The release date was converted into two numerical variables:

$$release_year_i \text{ and } release_month_i$$

This transformation allows the model to capture possible temporal patterns in game reception, such as differences between older and newer releases or seasonal release effects.

The variable `Estimated owners` was originally stored as a range, for example 0 - 20000. To make it usable as a numerical predictor, I converted each range into its midpoint:

$$owners_mid_i = \frac{owners_min_i + owners_max_i}{2}$$

where $owners_min_i$ and $owners_max_i$ are respectively the lower and upper bound of the ownership range.

I also reconstructed an estimate of the original price before discount. First, the discount variable was converted into a fractional value:

$$discount_frac_i = \frac{Discount_i}{100}$$

when discounts were expressed as percentages. Then, the original price was approximated as:

$$price_original_i = \frac{Price_i}{1 - discount_frac_i}$$

To avoid invalid values, discounts were clipped below 1, and infinite values were replaced with missing values and then filled using the observed current price.

The initial numerical feature set used in the baseline models was:

$$X_{num} = \{price_original, owners_mid, release_year, release_month, \\ Average\ playtime\ forever, Median\ playtime\ forever, Peak\ CCU\}$$

The variables `Positive`, `Negative` and `total_reviews` were excluded from the predictors because they are directly used to construct the target variable. Including them would create target leakage and would make the predictive task artificially easy.

4.2 Train-Test Split

After filtering the dataset to include only games with at least one review and after constructing the target variable, the data were split into training and test sets. I used an 80/20 train-test split with a fixed random seed:

$$80\% \text{ training data, } \quad 20\% \text{ test data}$$

The split was performed with `random_state = 42` to make the experiment reproducible. The resulting partitions contained 66,212 training observations and 16,554 test observations. The test set was kept fixed and was used only for final evaluation and comparison, while model selection and feature-set selection were performed on the training set through cross-validation.

4.3 Numerical Baseline Models

The first modelling step used only the numerical feature set. This allowed me to establish a baseline before adding richer categorical metadata. Four models were compared:

- **Dummy Regressor:** a naive baseline that always predicts the mean target value in the training set;
- **Ridge Regression:** a linear model with L_2 regularization;
- **Lasso Regression:** a linear model with L_1 regularization;
- **XGBoost Regressor:** a non-linear gradient boosting model based on decision trees.

For all numerical baseline models, missing numerical values were imputed using the median estimated from the corresponding training fold. Ridge and Lasso additionally included a `StandardScaler`, since regularized linear models are sensitive to the scale of the predictors. All preprocessing steps were included inside a scikit-learn `Pipeline` (Pedregosa et al., 2011),

ensuring that imputation and scaling were fitted only on the training data within each cross-validation fold. In the subsequent XGBoost models (Chen & Guestrin, 2016), any remaining missing numerical values were handled natively by the estimator.

The numerical models were tuned using `GridSearchCV` with 5-fold cross-validation and negative root mean squared error as the optimization criterion. The following hyperparameter grids were tested:

$$\alpha_{Ridge} \in \{0.1, 1, 10, 100\}$$

$$\alpha_{Lasso} \in \{0.001, 0.01, 0.1, 1\}$$

For the numerical-only XGBoost model, the grid was:

$$n_estimators \in \{100, 300\}$$

$$max_depth \in \{4, 6\}$$

$$learning_rate \in \{0.05, 0.1\}$$

This first comparison was used to understand whether the relationship between numerical metadata and user reception was mostly linear or whether a more flexible non-linear model was needed.

4.4 Categorical and Multilabel Encoding

After the numerical baseline, I extended the feature space by adding Steam categorical metadata. In particular, I used **Genres**, **Categories** and later **Tags**. These variables are multilabel categorical features, because each game can belong to more than one genre, category or tag.

Instead of applying a standard one-hot encoder to a single categorical value, I parsed each comma-separated field into a list of labels. For example, a game with:

Action, Adventure, Indie

was represented through separate binary indicators for each label. Each encoded variable takes value 1 if the game contains that label and 0 otherwise.

To avoid train-serving skew and target leakage, the set of labels was selected using only the training set. The same fixed label vocabulary was then applied to both the training and test sets. Rare labels were excluded to reduce noise and control the dimensionality of the feature space. Specifically:

- **Genres** and **Categories** were retained if they appeared at least 100 times in the training set;
- **Tags** were retained if they appeared at least 200 times in the training set.

This procedure produced three progressively richer feature blocks:

$$X_{num}$$

$$X_{num+cat} = X_{num} + \text{Genres} + \text{Categories}$$

$$X_{num+cat+tags} = X_{num} + \text{Genres} + \text{Categories} + \text{Tags}$$

The final feature matrices were checked to ensure that the training and test sets had exactly the same columns.

The frequency thresholds retained 24 genre labels, 49 category labels and 341 Steam Tags. Together with the seven numerical predictors, the final feature matrix contained 421 columns.

4.5 Feature Block Selection

To decide whether adding tags improved predictive performance, I compared the two categorical feature blocks using cross-validation on the training set only:

$$X_{num+cat} \quad \text{versus} \quad X_{num+cat+tags}$$

Both feature sets were evaluated using the same XGBoost configuration:

$$n_estimators = 300$$

$$max_depth = 4$$

$$learning_rate = 0.05$$

$$subsample = 0.8$$

$$colsample_bytree = 0.8$$

The comparison was performed with 3-fold cross-validation. The test set was not used for feature block selection, in order to preserve it as an unbiased final evaluation set.

4.6 Final Model Tuning

After selecting the best-performing feature block through cross-validation, I tuned the final XGBoost model using `GridSearchCV`. The final tuning grid was intentionally compact to keep the experiment computationally feasible while still testing different levels of model complexity:

$$n_estimators \in \{300\}$$

$$max_depth \in \{3, 4\}$$

$$learning_rate \in \{0.03, 0.05\}$$

$$subsample \in \{0.8\}$$

$$colsample_bytree \in \{0.8\}$$

The final model was trained with the squared error objective:

$$\text{objective} = \text{reg:squarederror}$$

and all stochastic components were controlled using `random_state = 42`.

The best configuration selected by cross-validation used `n_estimators = 300`, `max_depth = 4`, `learning_rate = 0.05`, `subsample = 0.8` and `colsample_bytree = 0.8`. The corresponding cross-validated RMSE was approximately 17.

The selected estimator was then retrained on the full training set and evaluated once on the held-out test set.

4.7 Evaluation Metrics

Model performance was evaluated using three regression metrics: Root Mean Squared Error, Mean Absolute Error and R^2 .

The Root Mean Squared Error is defined as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE penalizes large errors more heavily and is therefore useful for detecting models that make severe mistakes on some games.

The Mean Absolute Error is defined as:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE is easier to interpret because it measures the average absolute prediction error in the same units as the target variable.

Finally, R^2 measures the proportion of target variance explained by the model:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

A higher R^2 indicates that the model explains more variation in user reception compared to a simple mean prediction baseline.

4.8 Error Analysis and Model Interpretation

After evaluating the final model, I performed an error analysis to understand where the model performed well and where it failed. For each game in the test set, I computed the prediction error:

$$error_i = y_i - \hat{y}_i$$

and the absolute error:

$$abs_error_i = |y_i - \hat{y}_i|$$

The largest errors were inspected manually, and the test observations were also divided into quantile-based bins according to the true target value. This made it possible to check whether the model systematically overestimated poorly received games or underestimated highly received games.

For model interpretation, I computed permutation importance on a random sample of 5,000 observations from the held-out test set, using three permutation repeats. This method measures the increase in RMSE obtained when the values of one feature are randomly shuffled while all remaining predictors are left unchanged.

These interpretation tools were used only to understand predictive patterns. They should not be interpreted causally: a feature with high importance is useful for prediction, but this does not imply that it directly causes higher or lower user reception.

5 Results

This section reports the empirical results of the modelling pipeline. The evaluation follows the same sequence used in the notebook: first, different model families are compared using only numerical features; then, after selecting XGBoost as the strongest numerical model, progressively richer categorical feature blocks are added.

5.1 Numerical Baseline and Model Family Selection

The first comparison was performed using only numerical features. This step was used to evaluate whether regularized linear models were sufficient or whether a more flexible non-linear model was needed.

Model	CV RMSE	Test RMSE	Test MAE	Test R^2
Dummy mean	20.474	21.035	15.585	0.000
Ridge	19.609	20.277	14.744	0.070
Lasso	19.609	20.275	14.744	0.071
XGBoost	18.545	19.001	13.879	0.184

Table 1: Numerical-only model comparison.

The regularized linear models improve over the dummy baseline, but only moderately. Ridge and Lasso obtain very similar results, with a test R^2 of approximately 0.07. This suggests that linear relationships between the numerical metadata and the Bayesian-smoothed review score explain only a limited part of the variation in user reception.

XGBoost achieved the lowest cross-validated RMSE among the numerical models, with a value of 18.545, compared with approximately 19.609 for Ridge and Lasso. It was therefore selected as the modelling framework for the subsequent feature-extension steps. Its held-out test RMSE of 19.001 and test R^2 of 0.184 are reported for comparison. The performance gap between XGBoost and the regularized linear models suggests that the relationship between the numerical predictors and user reception contains relevant non-linearities.

5.2 Effect of Categorical Metadata

After selecting XGBoost as the strongest numerical model, I extended the feature space by adding multilabel categorical metadata. First, I added **Genres** and **Categories**. Then, I added **Steam Tags**, which provide a more granular description of gameplay style, mechanics, themes and audience expectations.

The two richer feature blocks were compared using cross-validation on the training set only. This ensured that the held-out test set was not used for feature selection.

Feature set	CV RMSE (mean $\pm$ SD)
Numerical + Genres/Categories	17.590 $\pm$ 0.042
Numerical + Genres/Categories/Tags	17.000 $\pm$ 0.046

Table 2: Three-fold cross-validated RMSE for the categorical feature blocks. Values are reported as mean $\pm$ standard deviation across folds.

Adding **Genres** and **Categories** improves performance compared to the numerical-only XGBoost model, showing that categorical metadata contains additional predictive information. The further improvement obtained by adding **Tags** suggests that user-defined metadata captures aspects of game identity that are not fully represented by broader genre and category labels.

5.3 Final Held-Out Test Performance

Based on the cross-validation results, the final model was trained using numerical features, **Genres**, **Categories** and **Tags**. The selected XGBoost configuration was then evaluated once on the held-out test set.

Model	RMSE	MAE	R^2
Dummy mean	21.035	15.585	0.000
XGBoost numeric only	19.001	13.879	0.184
XGBoost + Genres/Categories	17.956	13.105	0.271
Final XGBoost + Genres/Categories/Tags	17.383	12.618	0.317

Table 3: Held-out test performance of the main modelling stages.

The final model achieves the best held-out performance, with an RMSE of 17.379, an MAE of 12.608 and an R^2 of 0.317. Compared to the dummy baseline, the final model reduces RMSE by approximately 17.4%. Compared to the numerical-only XGBoost model, the addition of multilabel categorical metadata reduces RMSE by approximately 8.7%.

Overall, the results show that Steam metadata and post-release engagement indicators contain meaningful predictive signal for user reception. However, the moderate final R^2 also indicates that a substantial part of user reception remains unexplained by the available structured metadata alone.

6 Discussion

6.1 Interpretation of the Results

The results show that Steam metadata and post-release engagement signals contain a meaningful, but incomplete, predictive signal for user reception. The dummy baseline provides a useful reference point: predicting only the average Bayesian-smoothed review score leads to an RMSE of 21.035 and essentially no explanatory power. The improvement obtained by the numerical-only XGBoost model indicates that structured post-release variables such as ownership estimates, playtime indicators and peak concurrent users are informative about user reception.

At the same time, the comparison between linear models and XGBoost suggests that the relationship between these predictors and user reception is not purely linear. Ridge and Lasso improve only moderately over the dummy baseline when using numerical features, while XGBoost achieves a stronger performance. This is consistent with the idea that user reception may depend on non-linear interactions between popularity, engagement and game characteristics. For example, high playtime may be associated with positive reception for some types of games, but not necessarily for all games; similarly, a high number of owners may reflect broad visibility, but not always high satisfaction.

The progressive addition of categorical metadata further improves performance. Adding **Genres** and **Categories** reduces the prediction error compared to the numerical-only model, while the inclusion of **Tags** provides the strongest categorical improvement. This result is important because it suggests that user reception is not explained only by quantitative engagement signals. Community and platform metadata also capture relevant information about the type of experience offered by a game, its audience, its mechanics and its market positioning.

The final model achieves an RMSE of 17.379, an MAE of 12.608 and an R^2 of 0.317 on the held-out test set. These values indicate that the model captures part of the systematic variation in user reception, but also that a large share of the variation remains unexplained. In practical terms, the average absolute error of about 12.6 points means that predictions are reasonably informative at an aggregate level, but not precise enough to be interpreted as exact evaluations of individual games. Therefore, the model should be understood as a tool for identifying broad reception patterns rather than as a definitive scoring system.

6.2 Model Interpretation

To assess which variables contribute most to the predictive performance of the final model, I computed permutation importance on a random sample of 5,000 observations from the held-out test set. This method measures how much the model’s RMSE increases when the values of a feature are randomly shuffled while all other variables remain unchanged. Figure 3 reports the twenty most important predictors.

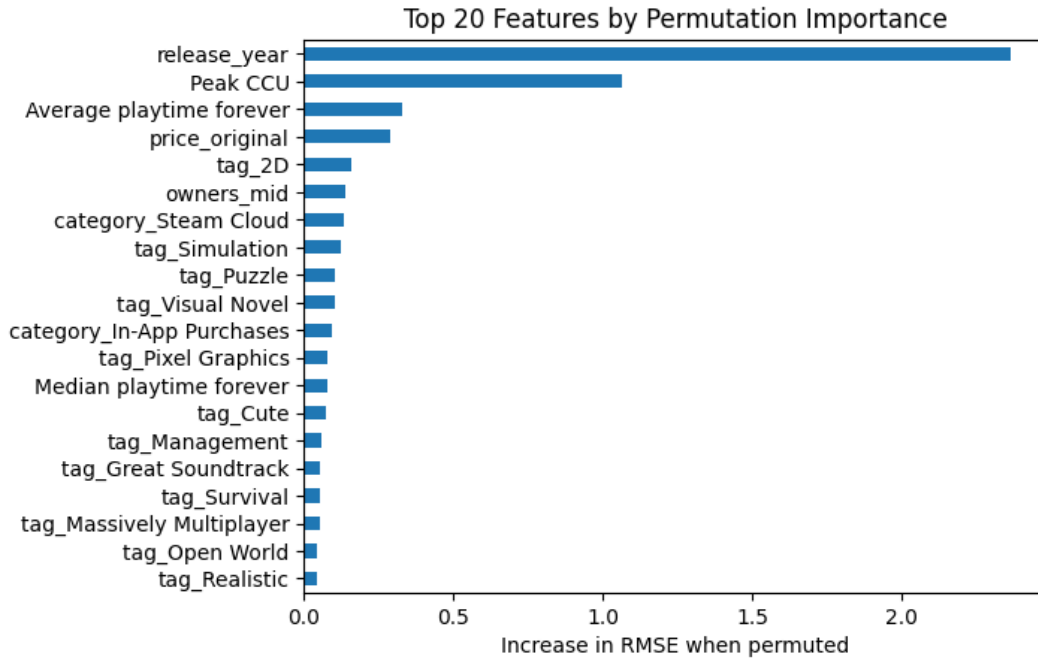


Figure 3: Top 20 features according to permutation importance for the final XGBoost model. Importance is expressed as the increase in RMSE obtained after permuting each feature on the held-out test sample.

As shown in Figure 3, `release_year` is the most influential predictor, followed by `Peak CCU`. Average playtime, the reconstructed original price and estimated ownership also contribute meaningfully to predictive performance. This confirms that post-release engagement and commercial metadata contain important information about user reception.

At the same time, several multilabel variables remain among the most relevant predictors, including `tag_2D`, `tag_Simulation`, `tag_Puzzle`, `tag_Visual Novel` and `category_Steam Cloud`. Their presence supports the feature-block comparison reported in the previous section: genres, categories and tags add information that is not fully captured by numerical indicators alone.¹

Tags provide a more granular description of each game than broad genres or categories. While a genre such as **Action** or **Adventure** describes a general market segment, tags can capture more specific characteristics such as gameplay mechanics, themes, difficulty, pacing, multiplayer structure or niche appeal. This additional information helps the model distinguish between games that may look similar in numerical terms but belong to different player communities or expectation profiles.

This is particularly relevant in the Steam ecosystem, where user reception is strongly shaped by audience expectations. A niche title can receive very positive reviews if it satisfies the expectations of a specific community, even if it has limited ownership or low peak concurrent

¹When predictors are correlated, predictive information maybe shared across multiple variables, which can reduce the measured permutation importance of each individual feature.

users. Conversely, a highly visible game can receive mixed or negative reviews if the player experience does not match expectations. Tags help capture part of this context, which explains why they improve predictive performance over numerical variables alone.

Permutation importance should be interpreted as a measure of predictive usefulness rather than as evidence of a causal effect. A high value means that shuffling the feature degrades the fitted model’s predictive performance; it does not imply that the feature directly causes better or worse user reception.

6.3 Error Analysis

The error analysis shows that the final model still tends to regress toward the mean of the target distribution. This means that the model is better at predicting games with average or moderately positive reception than games with extremely negative or extremely positive reception. Poorly received games tend to be overestimated, while highly received games tend to be underestimated.

Figure 4 compares the true Bayesian-smoothed review scores with the predictions generated by the final XGBoost model on the held-out test set. The dashed diagonal represents perfect predictions. The concentration of observations toward the center of the prediction range shows that the model tends to overestimate games with very low reception and underestimate games with very high reception.

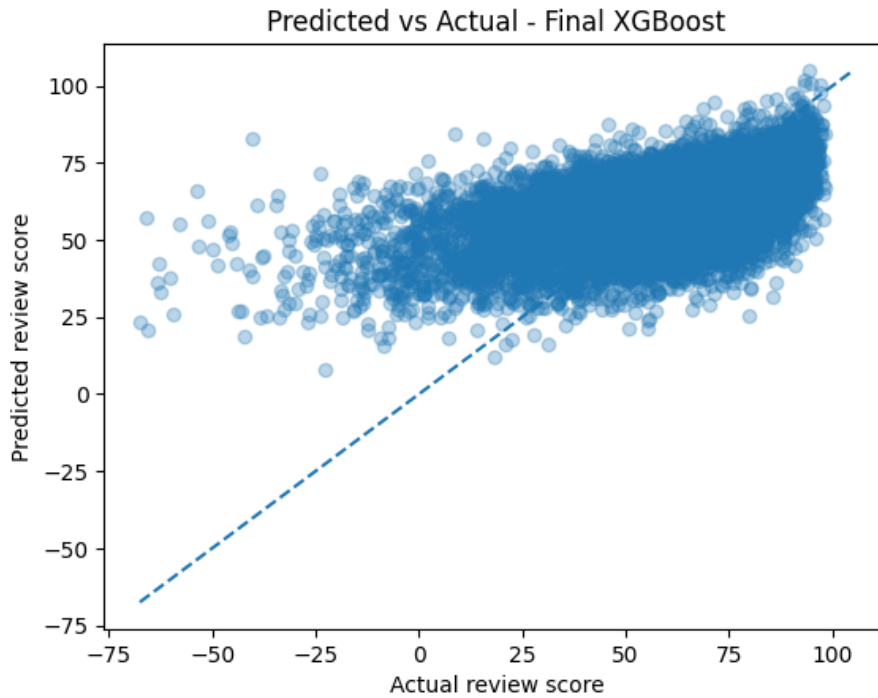


Figure 4: Predicted versus actual Bayesian-smoothed review scores for the final XGBoost model on the held-out test set.

Figure 5 quantifies the regression-to-the-mean pattern by reporting the mean signed prediction error across quintiles of the true target distribution. Since the error is defined as the actual score minus the predicted score, negative values indicate overestimation, while positive values indicate underestimation.

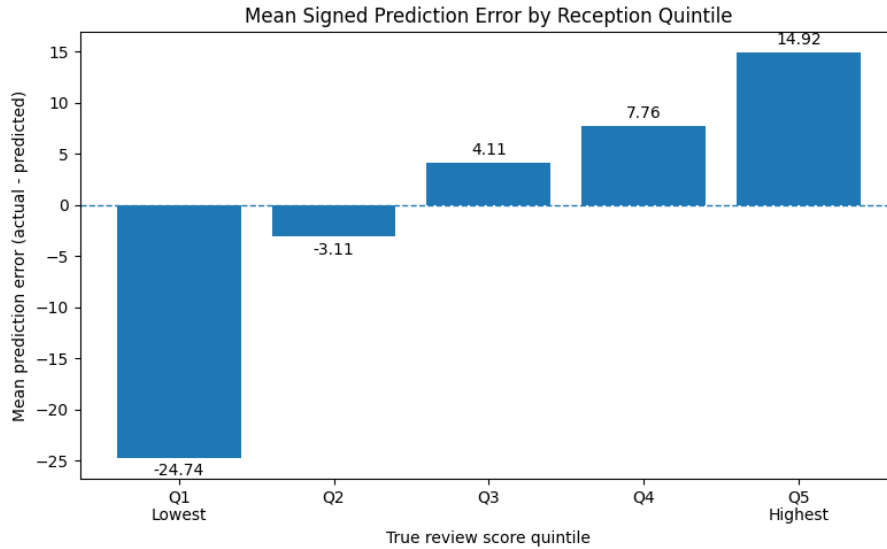


Figure 5: Mean signed prediction error across quintiles of the true Bayesian-smoothed review score. Negative values indicate overestimation, while positive values indicate underestimation.

As shown in Figure 5, the model substantially overestimates games in the lowest reception quintile, with a mean error of approximately -24.74 points. The bias becomes smaller in the second quintile and changes direction from the third quintile onwards. Games in the highest reception quintile are underestimated by approximately 14.92 points on average.

The mean predicted score varies from approximately 53.75 in the lowest quintile to only 69.49 in the highest quintile, whereas the corresponding mean true scores range from approximately 29.01 to 84.41 . This limited prediction range provides clear evidence that the model compresses outcomes toward the centre of the target distribution. Predictions are therefore more strongly biased at the two extremes and better centred for games in the middle of the distribution.

This pattern is common in regression problems where the most extreme outcomes depend on factors that are not fully observed in the structured feature set. In this project, the available predictors describe metadata, engagement and categorical positioning, but they do not directly measure many qualitative aspects that can strongly influence user reviews. These include technical issues, bugs at launch, quality of writing or gameplay design, monetization choices, update history, community controversies, developer reputation and the content of the reviews themselves.

The underestimation of highly received games suggests that exceptional positive reception may depend on qualities that are difficult to infer from metadata alone. For instance, a small independent game may obtain extremely positive reviews because of originality, emotional impact, strong community fit or excellent execution within a niche. These characteristics may not be fully captured by variables such as price, owners, playtime or tags.

Similarly, the overestimation of poorly received games suggests that negative reception may often be driven by specific problems that are not represented in the dataset. A game may have reasonable metadata and engagement signals but still receive poor reviews because of bugs, misleading marketing, performance problems, aggressive monetization or unmet expectations. Without review text or external community information, the model can only partially detect these cases.

Overall, the error analysis supports the main conclusion of the project: Steam metadata and post-release engagement indicators are informative, but they are not sufficient to fully explain user reception. The structured dataset captures broad and systematic patterns, while extreme reception appears to require richer qualitative and contextual information.

7 Limitations

This project has several limitations.

First, the model is predictive, not causal. Feature importance and permutation importance indicate which variables are useful for prediction, but they do not imply that a feature directly causes higher or lower user reception.

Second, this is not a pre-release forecasting model. Several predictors, such as estimated owners, playtime indicators and peak concurrent users, are available only after release. Therefore, the model should be interpreted as a post-release reception model rather than as a tool for predicting success before launch.

Third, the target variable is an NPS-inspired proxy, not a true Net Promoter Score. Steam reviews are binary positive/negative evaluations, while the traditional NPS is based on survey responses and distinguishes between promoters, passives and detractors. The constructed target is useful for modelling user reception, but it should not be interpreted as a direct measure of NPS.

Fourth, Steam reviews are self-selected. Users who decide to leave a review may not represent the entire player base, and review behaviour can be influenced by strong positive or negative experiences, community campaigns, controversies or review bombing.

Fifth, the dataset does not include several qualitative and contextual variables that may be highly relevant for explaining user reception. These include review text, marketing exposure, wishlist data, update history, technical issues, bug reports, external social media signals and developer reputation.

Sixth, the random train-test split measures cross-sectional generalization within the observed Steam catalogue rather than out-of-time performance on future releases. Since `release_year` is the most influential predictor, a chronological hold-out set would be necessary to assess temporal generalization and robustness to changes in the Steam market.

Seventh, Steam Tags are informative but potentially noisy. They may reflect user perception, platform conventions or community labelling dynamics rather than stable and objective game characteristics.

Finally, games with no user reviews were excluded because their target could not be constructed. Consequently, the results apply only to games with an observable review signal and may not generalize to the least visible titles in the Steam catalogue.

8 Conclusion

This project investigated whether Steam game metadata and post-release engagement signals can be used to predict user reception. Starting from a cleaned version of the Steam Games Dataset, I constructed an NPS-inspired Bayesian-smoothed review score based on positive and negative user reviews. This target was designed to reduce the instability of raw review ratios, especially for games with very few reviews.

The results show that structured Steam data contains meaningful predictive signal. Regularized linear models improve only moderately over the dummy baseline, while XGBoost achieves stronger performance, suggesting that the relationship between metadata, engagement and reception is partly non-linear. The progressive addition of multilabel metadata further improves performance, with Tags providing the strongest categorical contribution.

The final XGBoost model, trained on numerical features, Genres, Categories and Tags, achieves an RMSE of approximately 17.38, an MAE of approximately 12.61 and an R^2 of 0.317 on the held-out test set. These results indicate that the model can capture broad patterns in user reception, but also that a substantial part of the variation remains unexplained.

Overall, the answer to the research question is partially positive: Steam metadata and post-release engagement indicators can be used to predict user reception to a meaningful extent,

but they are not sufficient to fully explain extreme positive or negative outcomes. Future work could improve the model by incorporating review text, update history, bug-related information, developer reputation and external community sentiment.

Beyond the technical scope of this report, the incremental improvement obtained from Steam Tags suggests a possible connection between user reception and product differentiation on Steam.

Data and Code Availability

The cleaned dataset used in this project is publicly available on Kaggle. The analysis was conducted in Google Colab, and the complete notebook is available on GitHub.

References

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). doi:10.1145/2939672.2939785

IMDb. (n.d.). *Ratings FAQ*. Accessed July 2026. <https://help.imdb.com/article/imdb/track-movies-tv/ratings-faq/G67Y87TFYYP6TWAV/>

Miller, E. (2009). *How Not to Sort by Average Rating*. <https://www.evanmiller.org/how-not-to-sort-by-average-rating.html>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>

Reichheld, F. F. (2003). The one number you need to grow. *Harvard Business Review*, 81(12), 46–54.

Roman, M. B. (2022). *Steam Games Dataset* [Dataset]. Kaggle. doi:10.34740/KAGGLE/DS/2109585

Triscali, S. (2026). *Steam Dataset 2026 Cleaned* [Dataset]. Kaggle. doi:10.34740/KAGGLE/DS/9847288